

# SynchroKin Developer Guide

## 11-Node Full-Body Hardware Replication & Firmware

### Contents

---

|          |                                                           |          |
|----------|-----------------------------------------------------------|----------|
| <b>1</b> | <b>Hardware Fabrication</b>                               | <b>1</b> |
| 1.1      | 3D Casing (.ipt files) & Labeling . . . . .               | 1        |
| 1.2      | Circuit Wiring . . . . .                                  | 2        |
| <b>2</b> | <b>Establishing a New Tracking Module</b>                 | <b>3</b> |
| <b>3</b> | <b>Firmware Source Code</b>                               | <b>4</b> |
| 3.1      | Universal Leaf Node Firmware (Limb IDs 1 to 10) . . . . . | 4        |
| 3.2      | Sub-Master Relay Hub (Alpha Code) . . . . .               | 6        |
| 3.3      | Master USB Receiver Code (11-Node Scaled) . . . . .       | 9        |

## 1 Hardware Fabrication

---

To replicate a SynchroKin tracking node, you must first construct the physical hardware module. The system's ESP-NOW Relay architecture natively supports up to 11 simultaneous nodes (Alpha (chest) + 10).

### 1.1 3D Casing (.ipt files) & Labeling

The external enclosures are designed in Autodesk Inventor. The source .ipt CAD files and the exported .stl files for 3D printing are hosted in the CAD subfolder of this repository. You can download the 3D modeling package directly here:

[\[Download SynchroKin .ipt Files\]](#) Print these using standard PLA or PETG with 20% infill for durability.

**Best Practice Advice:** Immediately after printing, assembling, and flashing a new module, physically stick a label on the casing with the module's Greek identifier (e.g., "Zeta", "Eta"). Because all nodes look physically identical, this prevents catastrophic mapping errors during clinical patient setup.



Figure 1: 3D Casing Design for the Wearable Node PLEASE ATTACH THE BNO055 IN THIS EXACT ORIENTATION (IF NEEDED PLEASE CHECK THE BETA or GAMMA MODULE).

### 1.2 Circuit Wiring

Wire the components according to the schematic below. Ensure the custom voltage divider (two  $100\text{k}\Omega$  resistors) is securely soldered to the battery line and routed to GPIO 2 (ADC) to enable battery monitoring.

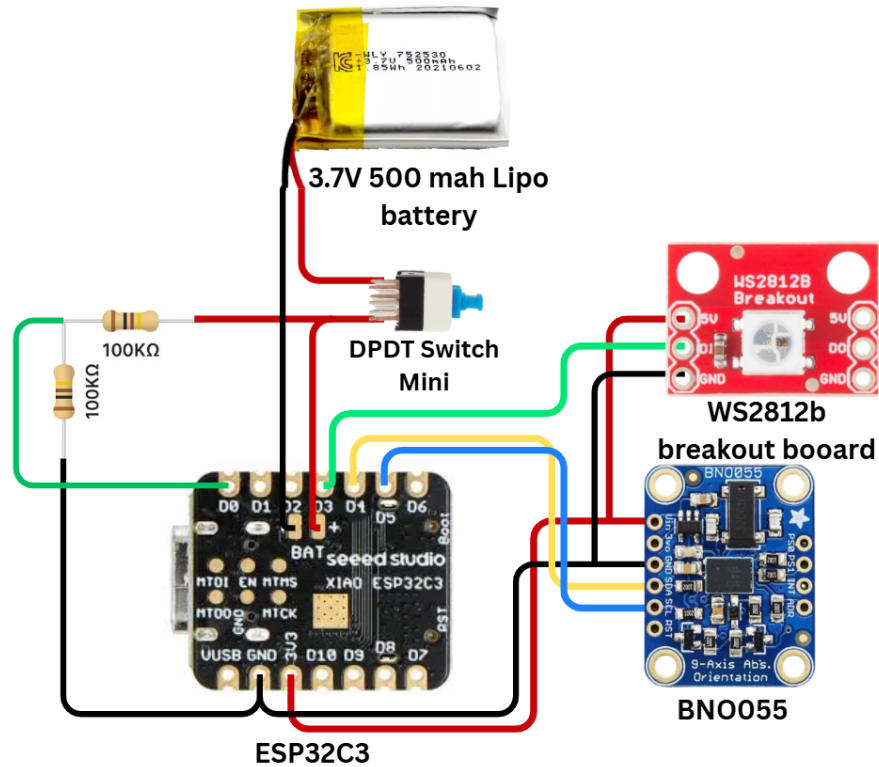


Figure 2: Circuit Schematic for individual wireless nodes.



Figure 3: Antenna Attachment

## 2 Establishing a New Tracking Module

---

Adding a new node (e.g., "Zeta") to the ecosystem is a standardized process:

1. **Fabricate & Label:** Build the hardware and label it "Zeta".
2. **Assign ID:** Open the *Universal Leaf Node Firmware* (Section 3.1). Scroll to MY\_LIMB\_ID and change it to an unused integer between 1 and 10 (e.g., 5 for Zeta).
3. **Flash:** Upload the code to the new module.

4. **Verify:** Because the Alpha and Master nodes provided in this document are pre-programmed with arrays supporting IDs 1 through 10, the new node will automatically be detected and routed to Unity.

### 3 Firmware Source Code

---

Below is the heavily commented, 11-node production firmware. **Important Note on Data Limits:** The ESP-NOW maximum payload is 250 bytes. Sending 11 quaternions (176 bytes) plus a 16-bit status mask (2 bytes) totals 178 bytes, functioning perfectly within protocol limits.

#### 3.1 Universal Leaf Node Firmware (Limb IDs 1 to 10)

This code is flashed to any peripheral tracking module. [View the Universal Leaf Node Firmware on GitHub](#)

```
1 #include <Wire.h>
2 #include <WiFi.h>
3 #include <esp_now.h>
4 #include <Adafruit_NeoPixel.h>
5 #include <Adafruit_Sensor.h>
6 #include <Adafruit_BNO055.h>
7 #include <utility/imuMaths.h>
8
9 // =====
10 // 1. MAC ADDRESS CONFIGURATION
11 // =====
12 // This must be the MAC address of your ALPHA (Chest) Relay Hub.
13 uint8_t alphaAddress[] = {0x1C, 0xDB, 0xD4, 0xED, 0x09, 0xFC};
14
15 // =====
16 // 2. NODE IDENTITY SETUP
17 // =====
18 // Assign a unique ID before flashing to a new board:
19 // 1=Beta, 2=Gamma, 3=Delta, 4=Epsilon, 5=Zeta, 6=Eta, 7=Theta, 8=Iota, 9=Kappa, 10=
   Lambda
20 const uint8_t MY_LIMB_ID = 1;
21
22 // =====
23 // 3. HARDWARE PINS & SETUP
24 // =====
25 const int LED_PIN = 2;
26 #define BATTERY_PIN 2 // ADC pin reading the voltage divider
27 #define RGB_PIN 5 // Data pin for WS2812B NeoPixel
28 #define NUMPIXELS 1
29
30 Adafruit_NeoPixel pixels(NUMPIXELS, RGB_PIN, NEO_GRB + NEO_KHZ800);
31 Adafruit_BNO055 bno = Adafruit_BNO055(55, 0x28, &Wire);
32
33 // =====
34 // 4. SYSTEM STATUS VARIABLES
35 // =====
36 bool isConnected = false;
37 float batVoltage = 4.2;
38 unsigned long lastBatCheck = 0;
39 unsigned long lastBlinkTime = 0;
40 unsigned long lastSuccessfulSend = 0;
41
42 typedef struct limb_message {
43     uint8_t limbID;
44     float qW; float qX; float qY; float qZ;
45 } limb_message;
46 limb_message myData;
47
48 esp_now_peer_info_t peerInfo;
49 unsigned long lastPrintTime = 0;
50
51 // =====
52 // 5. ESP-NOW TRANSMIT CALLBACK
```

```

53 // =====
54 void OnDataSent(const wifi_tx_info_t *info, esp_now_send_status_t status) {
55     if (status == ESP_NOW_SEND_SUCCESS) {
56         lastSuccessfulSend = millis();
57     }
58 }
59
60 void setup() {
61     Serial.begin(115200);
62     Wire.begin(); Wire.setClock(400000);
63
64     pixels.begin(); pixels.setBrightness(50); pixels.clear(); pixels.show();
65     pinMode(BATTERY_PIN, INPUT); pinMode(LED_PIN, OUTPUT); digitalWrite(LED_PIN, LOW);
66
67     WiFi.mode(WIFI_STA);
68     if (esp_now_init() != ESP_OK) return;
69     esp_now_register_send_cb(OnDataSent);
70
71     memcpy(peerInfo.peer_addr, alphaAddress, 6);
72     peerInfo.channel = 0; peerInfo.encrypt = false;
73     esp_now_add_peer(&peerInfo);
74
75     if (!bno.begin()) {
76         while (1) { // Hardware failure lock
77             pixels.setPixelColor(0, pixels.Color(255, 255, 0)); pixels.show(); delay(500);
78             pixels.setPixelColor(0, pixels.Color(0, 0, 0)); pixels.show(); delay(500);
79         }
80     }
81     delay(100); bno.setExtCrystalUse(true); digitalWrite(LED_PIN, HIGH);
82
83     myData.limbID = MY_LIMB_ID;
84     lastPrintTime = micros();
85 }
86
87 void loop() {
88     unsigned long nowMillis = millis();
89     unsigned long now = micros();
90
91     isConnected = (nowMillis - lastSuccessfulSend < 500);
92
93     // --- BATTERY LOGIC ---
94     if (nowMillis - lastBatCheck >= 1000) {
95         lastBatCheck = nowMillis;
96         long sum = 0;
97         for(int i = 0; i < 20; i++) { sum += analogRead(BATTERY_PIN); delay(1); }
98         batVoltage = ((float)sum / 20.0 / 4095.0) * 3.3 * 2.0;
99     }
100
101     // --- DIAGNOSTIC LED LOGIC ---
102     if (batVoltage < 3.2) {
103         pixels.setPixelColor(0, pixels.Color(255, 0, 0));
104     } else {
105         uint32_t baseColor = isConnected ? pixels.Color(0, 0, 255) : pixels.Color(0, 255,
106         0);
107         if (batVoltage >= 3.2 && batVoltage < 3.6) {
108             if (nowMillis - lastBlinkTime > 1500) lastBlinkTime = nowMillis;
109             if (nowMillis - lastBlinkTime > 1000) pixels.setPixelColor(0, pixels.Color
110             (255, 0, 0));
111             else pixels.setPixelColor(0, baseColor);
112         } else {
113             pixels.setPixelColor(0, baseColor);
114         }
115     }
116     pixels.show();
117
118     // --- TRANSMISSION LOGIC (100Hz) ---
119     if (now - lastPrintTime >= 10000) {
120         lastPrintTime = now;
121         imu::Quaternion quat = bno.getQuat();
122         myData.qW = quat.w(); myData.qX = quat.x(); myData.qY = quat.y(); myData.qZ = quat.z
123         ();
124         esp_now_send(alphaAddress, (uint8_t *) &myData, sizeof(myData));
125     }

```

Listing 1: Universal Leaf Node Firmware

### 3.2 Sub-Master Relay Hub (Alpha Code)

This node gathers incoming packets from up to 10 leaf modules, concatenates them into a 2D array, and fires the 178-byte Fat Payload to the Master. [View the Alpha \(Chest\) Sub-Master Relay Firmware on GitHub](#)

```

1  #include <Wire.h>
2  #include <WiFi.h>
3  #include <esp_now.h>
4  #include <Adafruit_NeoPixel.h>
5  #include <Adafruit_Sensor.h>
6  #include <Adafruit_BNO055.h>
7  #include <utility/ImuMaths.h>
8
9  // =====
10 // 1. MAC ADDRESS CONFIGURATION
11 // =====
12 uint8_t masterAddress[] = {0xD4, 0xE9, 0xF4, 0xB1, 0x83, 0xA4};
13
14 const int LED_PIN = 2;
15 #define BATTERY_PIN 2
16 #define RGB_PIN 5
17 #define NUMPIXELS 1
18
19 Adafruit_NeoPixel pixels(NUMPIXELS, RGB_PIN, NEO_GRB + NEO_KHZ800);
20 Adafruit_BNO055 bno = Adafruit_BNO055(55, 0x28, &Wire);
21
22 bool isConnectedToMaster = false;
23 float batVoltage = 4.2;
24 unsigned long lastBatCheck = 0;
25 unsigned long lastBlinkTime = 0;
26 unsigned long lastSuccessfulSend = 0;
27
28 // =====
29 // 3. 11-NODE DATA STRUCTURES
30 // =====
31 typedef struct limb_message {
32     uint8_t limbID;
33     float qW; float qX; float qY; float qZ;
34 } limb_message;
35 limb_message incomingLimbData;
36
37 // Fat payload structure. Uses a 2D array [11 nodes][4 quat values]
38 // Index 0 = Alpha. Index 1-10 = Peripheral Leaves.
39 typedef struct master_message {
40     float quats[11][4];
41     uint16_t activeNodes; // Upgraded to 16-bit to hold 11 status flags
42 } master_message;
43 master_message payload;
44
45 esp_now_peer_info_t peerInfo;
46
47 // Heartbeat trackers for 10 peripheral nodes
48 unsigned long leafBeats[11] = {0};
49 unsigned long lastPrintTime = 0;
50
51 // =====
52 // 4. ESP-NOW CALLBACKS
53 // =====
54 void OnDataSent(const wifi_tx_info_t *info, esp_now_send_status_t status) {
55     if (status == ESP_NOW_SEND_SUCCESS) lastSuccessfulSend = millis();
56 }
57
58 // Routes incoming data into the correct array slot based on ID
59 void OnDataRecv(const esp_now_recv_info_t *info, const uint8_t *incomingDataBytes, int
    len) {
60     memcpy(&incomingLimbData, incomingDataBytes, sizeof(incomingLimbData));

```



```
61  uint8_t id = incomingLimbData.limbID;
62
63  if (id >= 1 && id <= 10) {
64      payload.quats[id][0] = incomingLimbData.qW;
65      payload.quats[id][1] = incomingLimbData.qX;
66      payload.quats[id][2] = incomingLimbData.qY;
67      payload.quats[id][3] = incomingLimbData.qZ;
68      leafBeats[id] = millis();
69  }
70 }
71
72 void setup() {
73     Serial.begin(115200);
74     Wire.begin(); Wire.setClock(400000);
75
76     pixels.begin(); pixels.setBrightness(50); pixels.clear(); pixels.show();
77     pinMode(BATTERY_PIN, INPUT); pinMode(LED_PIN, OUTPUT); digitalWrite(LED_PIN, LOW);
78
79     WiFi.mode(WIFI_STA);
80     if (esp_now_init() != ESP_OK) return;
81
82     esp_now_register_send_cb(OnDataSent);
83     esp_now_register_recv_cb(OnDataRecv);
84
85     memcpy(peerInfo.peer_addr, masterAddress, 6);
86     peerInfo.channel = 0; peerInfo.encrypt = false;
87     esp_now_add_peer(&peerInfo);
88
89     if (!bno.begin()) {
90         while (1) {
91             pixels.setPixelColor(0, pixels.Color(255, 255, 0)); pixels.show(); delay(500);
92             pixels.setPixelColor(0, pixels.Color(0, 0, 0)); pixels.show(); delay(500);
93         }
94     }
95     delay(100); bno.setExtCrystalUse(true); digitalWrite(LED_PIN, HIGH);
96     lastPrintTime = micros();
97 }
98
99 void loop() {
100     unsigned long nowMillis = millis();
101     unsigned long now = micros();
102
103     isConnectedToMaster = (nowMillis - lastSuccessfulSend < 500);
104
105     // --- BATTERY & LED LOGIC ---
106     if (nowMillis - lastBatCheck >= 1000) {
107         lastBatCheck = nowMillis;
108         long sum = 0;
109         for(int i = 0; i < 20; i++) { sum += analogRead(BATTERY_PIN); delay(1); }
110         batVoltage = ((float)sum / 20.0 / 4095.0) * 3.3 * 2.0;
111     }
112
113     if (batVoltage < 3.2) {
114         pixels.setPixelColor(0, pixels.Color(255, 0, 0));
115     } else {
116         uint32_t baseColor = isConnectedToMaster ? pixels.Color(0, 0, 255) : pixels.Color
117         (0, 255, 0);
118         if (batVoltage >= 3.2 && batVoltage < 3.6) {
119             if (nowMillis - lastBlinkTime > 1500) lastBlinkTime = nowMillis;
120             if (nowMillis - lastBlinkTime > 1000) pixels.setPixelColor(0, pixels.Color
121             (255, 0, 0));
122             else pixels.setPixelColor(0, baseColor);
123         } else {
124             pixels.setPixelColor(0, baseColor);
125         }
126     }
127     pixels.show();
128
129     // --- SENSOR READING & TRANSMISSION (100Hz) ---
130     if (now - lastPrintTime >= 10000) {
131         lastPrintTime = now;
132         // Read local Chest BNO055 into Index 0
```

```
132     imu::Quaternion quat = bno.getQuat();
133     payload.quats[0][0] = quat.w(); payload.quats[0][1] = quat.x();
134     payload.quats[0][2] = quat.y(); payload.quats[0][3] = quat.z();
135
136     // Construct 16-bit status mask
137     payload.activeNodes = 1; // Alpha (Bit 0) is always 1
138     for (int i = 1; i <= 10; i++) {
139         if (nowMillis - leafBeats[i] < 500) {
140             payload.activeNodes |= (1 << i);
141         }
142     }
143
144     // Send the fat payload to the Master
145     esp_now_send(masterAddress, (uint8_t *) &payload, sizeof(payload));
146 }
147 }
```

Listing 2: Alpha (Chest) Sub-Master Relay Firmware



### 3.3 Master USB Receiver Code (11-Node Scaled)

Because we are printing 11 full quaternions over serial, we will quickly hit a buffer limit at standard baud rates. **CRITICAL:** This Master node uses a baud rate of 921600. You **MUST** update the Unity SynchroKin.cs baud rate property to match this, or Unity will freeze. [View the Master USB Receiver Firmware on GitHub](#)

```

1 #include <esp_now.h>
2 #include <WiFi.h>
3
4 const int BUILTIN_LED = 2;
5
6 // =====
7 // 1. DATA STRUCTURE (MUST MATCH ALPHA)
8 // =====
9 typedef struct master_message {
10     float quats[11][4];
11     uint16_t activeNodes;
12 } master_message;
13 master_message incomingData;
14
15 volatile bool newDataReady = false;
16
17 // Array of prefixes for the Unity String Parser
18 const char* nodePrefixes[11] = {
19     "A_", "B_", "G_", "D_", "E_", "Z_", "H_", "T_", "I_", "K_", "L_"
20 };
21
22 // =====
23 // 2. ESP-NOW RECEIVE CALLBACK
24 // =====
25 void OnDataRecv(const esp_now_recv_info_t *info, const uint8_t *incomingDataBytes, int
    len) {
26     if (len == sizeof(incomingData)) {
27         memcpy(&incomingData, incomingDataBytes, sizeof(incomingData));
28         newDataReady = true;
29     }
30 }
31
32 void setup() {
33     // CRITICAL: Baud rate upgraded to 921600 to support 11-node data density.
34     // Unity SynchroKin.cs MUST be updated to 921600.
35     Serial.begin(921600);
36
37     pinMode(BUILTIN_LED, OUTPUT);
38     digitalWrite(BUILTIN_LED, LOW);
39
40     WiFi.mode(WIFI_STA);
41     if (esp_now_init() != ESP_OK) return;
42     esp_now_register_recv_cb(OnDataRecv);
43 }
44
45 unsigned long lastMasterPrint = 0;
46
47 void loop() {
48     if (newDataReady) {
49         newDataReady = false;
50     }
51
52     // Throttle USB serial output to 50fps (every 20ms)
53     if (millis() - lastMasterPrint >= 20) {
54         lastMasterPrint = millis();
55
56         if (Serial && Serial.availableForWrite() > 256) {
57
58             // Print Status Watchdog line
59             Serial.print("STAT:");
60             for (int i = 0; i < 11; i++) {
61                 int stat = (incomingData.activeNodes & (1 << i)) ? 1 : 0;
62                 Serial.print(stat);
63                 if (i < 10) Serial.print(",");
64             }

```

```
65
66     // Dynamically print all 11 nodes for the Unity Parser
67     for (int i = 0; i < 11; i++) {
68         Serial.print(" "); Serial.print(nodePrefixes[i]); Serial.print("W:"); Serial.
print(incomingData.quats[i][0], 4);
69         Serial.print(" "); Serial.print(nodePrefixes[i]); Serial.print("X:"); Serial.
print(incomingData.quats[i][1], 4);
70         Serial.print(" "); Serial.print(nodePrefixes[i]); Serial.print("Y:"); Serial.
print(incomingData.quats[i][2], 4);
71         Serial.print(" "); Serial.print(nodePrefixes[i]); Serial.print("Z:");
72
73         if (i == 10) Serial.println(incomingData.quats[i][3], 4); // Newline on the
last node
74         else Serial.print(incomingData.quats[i][3], 4);
75     }
76 }
77 }
78 }
```

Listing 3: Master USB Receiver Firmware